

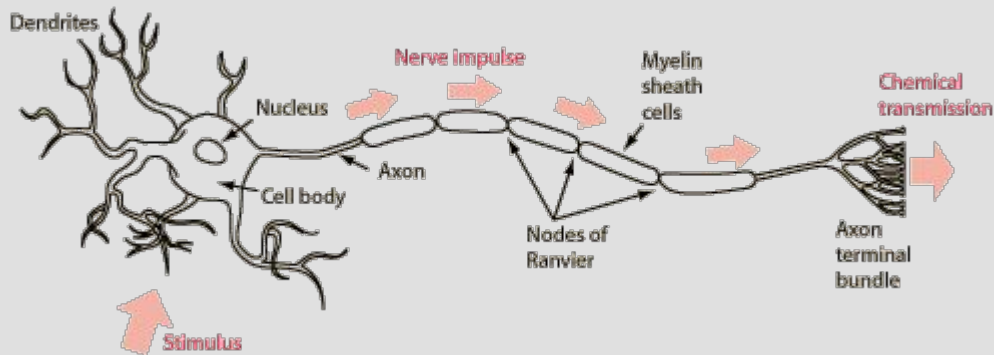
The Perceptron



CSCE 5218 / CSCE 4930 Deep Learning
Lecture 2

Biological Motivation

- The basic computational unit of the brain is a **neuron**
 - the human nervous system has as many as 86 billion neurons
- Given the stimuli, a neuron produces as output
 - electrical signals, range from 0 to 1 (both stimulus and outputs)
 - We like binary, so
 - 1 fire
 - 0 do not fire



Biological Motivation

- In the brain, neurons become connected to other neurons
 - the strength of the connection adapts over time (that's how humans learn)
- Neurons adapt to other functions if not used
 - Babies can differentiate sounds in all languages. As they grow, they lose the ability to distinguish sounds that are not relevant to the languages they speak
 - if you never hear it, why bother?

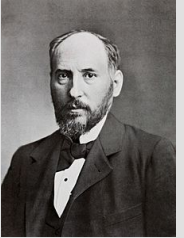
Santiago Ramón y Cajal (1852-1934)

- Inventor of the *Neuron Doctrine*

Neuron doctrine

From Wikipedia, the free encyclopedia

The **neuron doctrine** is the concept that the [nervous system](#) is made up of discrete individual cells, a discovery due to decisive neuro-anatomical work of [Santiago Ramón y Cajal](#) and later presented by, among others, [H. Waldeyer-Hartz](#).^[1] The term *neuron* (spelled *neurone* in British English) was itself coined by Waldeyer as a way of identifying the cells in question. The *neuron doctrine*, as it became known, served to position neurons as special cases under the broader [cell theory](#) evolved some decades earlier. He appropriated the concept not from his own research but from the disparate observation of the histological work of [Albert von Kölliker](#), [Camillo Golgi](#), [Franz Nissl](#), [Santiago Ramón y Cajal](#), [Auguste Forel](#) and others.^{[2][3]}



- Received the Nobel Prize in 1906 (with Camillo Golgi)

Santiago Ramón y Cajal (1852-1934)

Llinás, R.R., 2003. The contribution of Santiago Ramon y Cajal to functional neuroscience. *Nature Reviews Neuroscience*, 4(1), pp.77-80.

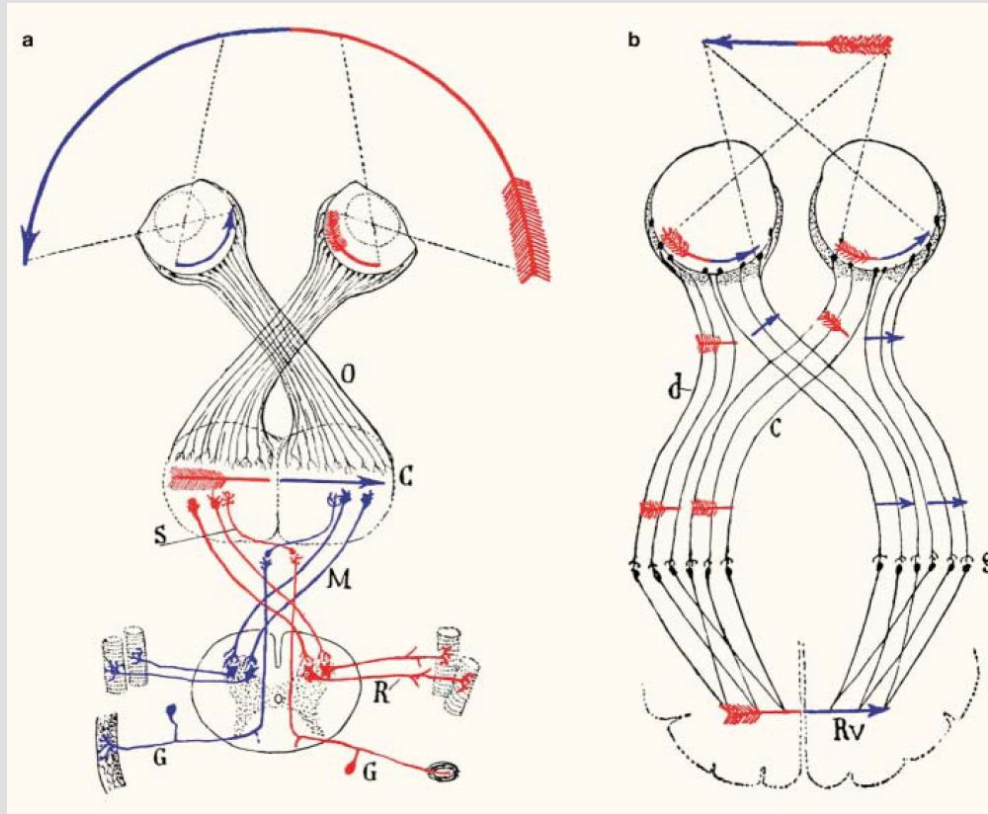
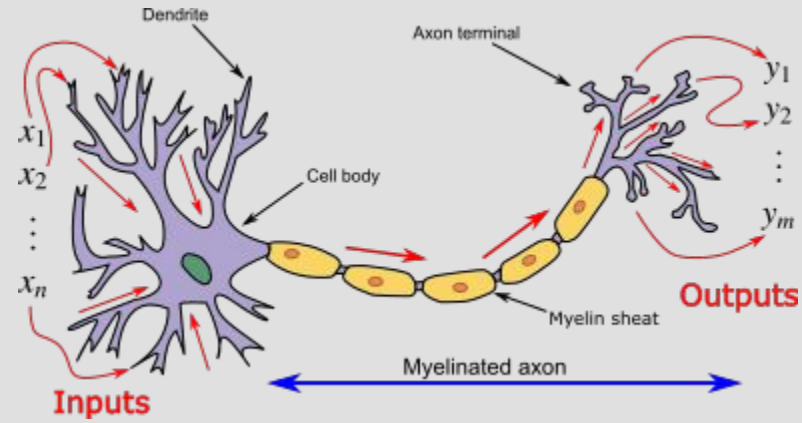


Figure 1 | **The organization of the central visual pathways.** **a** | The visual system of a lower vertebrate. The lateral location of the eyes in the head is such that the visual field overlap is small. The image must be continuous over the left/right axis of the animal in spite of the inversion generated by the optical properties of the eye. This inversion of the image is corrected, Cajal noted, by the total crossing of the optic nerve (O) and its orderly termination on the optic tectum (C). Both sensory (G and S) and motor (M and R) pathways from each side — red and blue — are also crossed such that all sensory inputs may be congruous in the generation of a global sensory image used for the activation of the motor system of each side. **b** | The visual system in humans. The frontal location of the eyes results in a large overlap of the visual image in both eyes. In this case, the crossing of the optic nerve occurs only for the nasal half of each retina (c) whereas the temporal half projects directly to the same side (d). The thalamocortical organization is such that corresponding points in each retina must terminate near each other at the visual cortex (Rv). This internal congruence in visual space is essential for the exquisite eye-hand coordination that is found in humans. g, neurons of the lateral geniculate nucleus. Modified, with permission, from the Instituto de Neurobiología "Ramón y Cajal", Madrid, Spain.

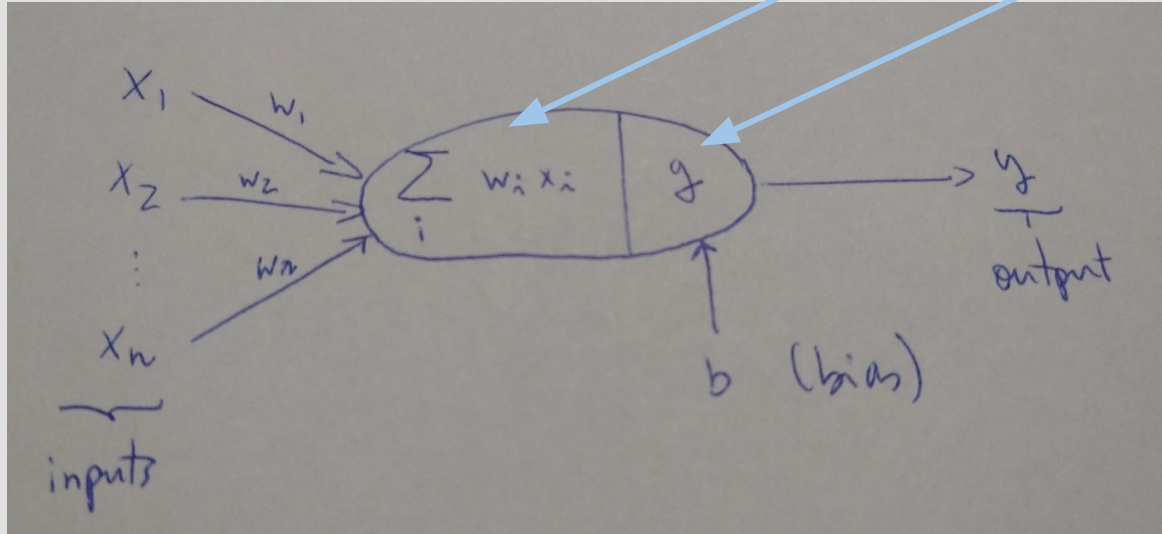
Biological Neuron Model

https://en.wikipedia.org/wiki/Biological_neuron_model



Mathematical Model - The Perceptron

- A simplified model of a biological neuron

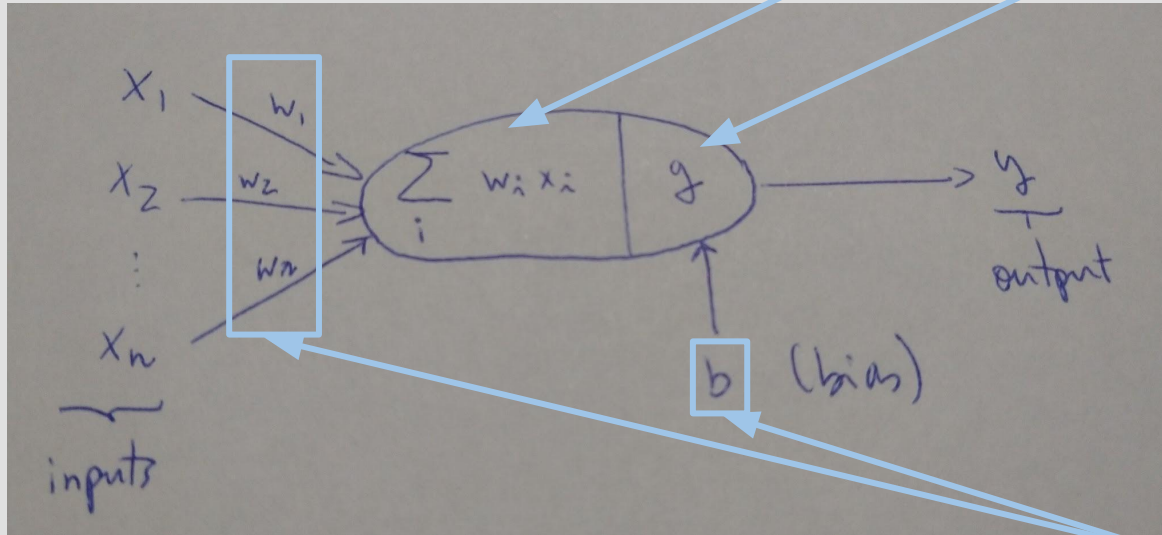


$$g(x) = \begin{cases} 1 & \text{if } x > b \\ 0 & \text{otherwise} \end{cases}$$

- The activation function ($g()$) can be many functions
 - idea: have a function that fires only when needed

Mathematical Model - The Perceptron

- A simplified model of a biological neuron



$$g(x) = \begin{cases} 1 & \text{if } x > b \\ 0 & \text{otherwise} \end{cases}$$

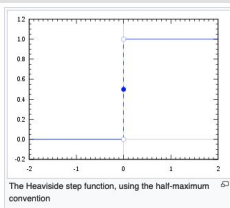
- The activation function ($g()$) can be many functions
 - idea: have a function that fires only when needed

Mathematical Model - The Perceptron

- Activation functions

- Step function

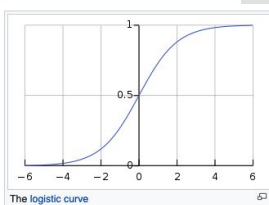
$$\begin{aligned} &1 \text{ if } x > 0 \\ &0 \text{ if } x < 0 \end{aligned}$$



A step function that fires above a certain value is called the *threshold function*

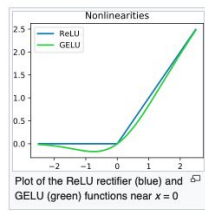
- Sigmoid Functions

$$S(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1}.$$



- ReLU

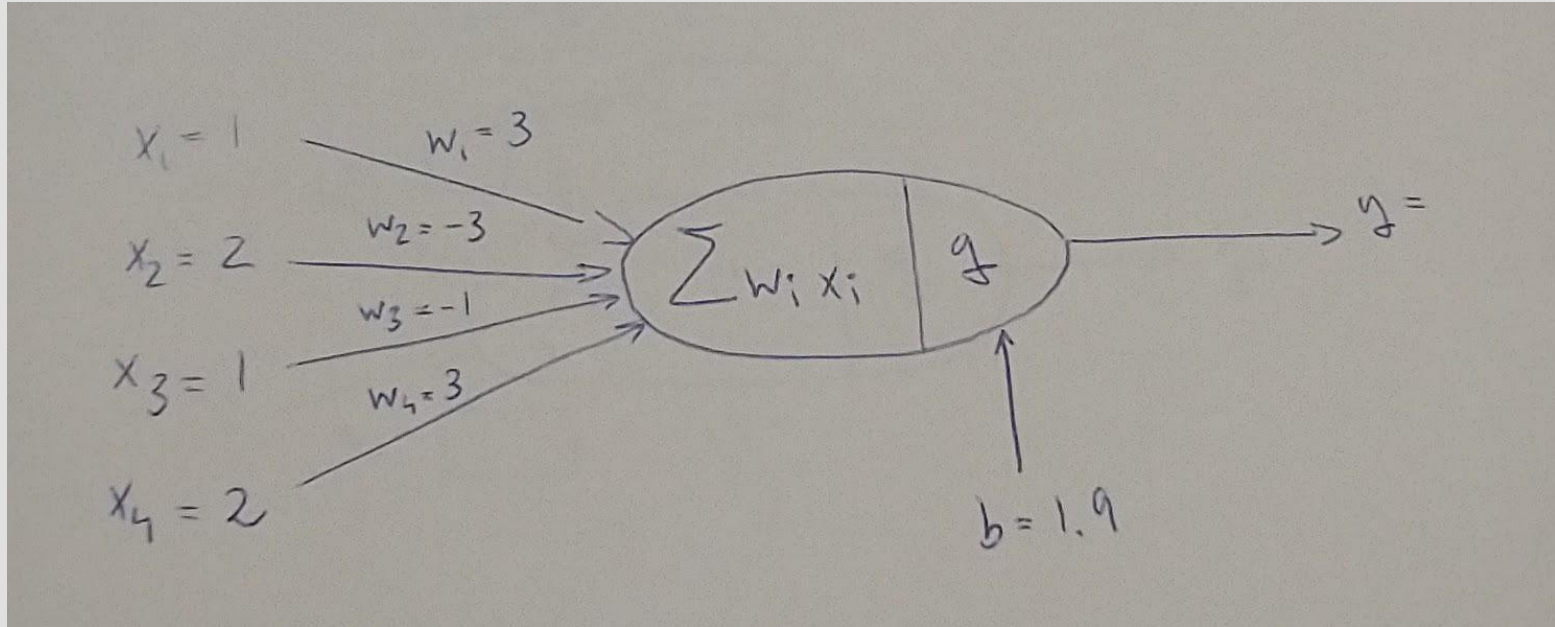
$$f(x) = x^+ = \max(0, x)$$



- and others

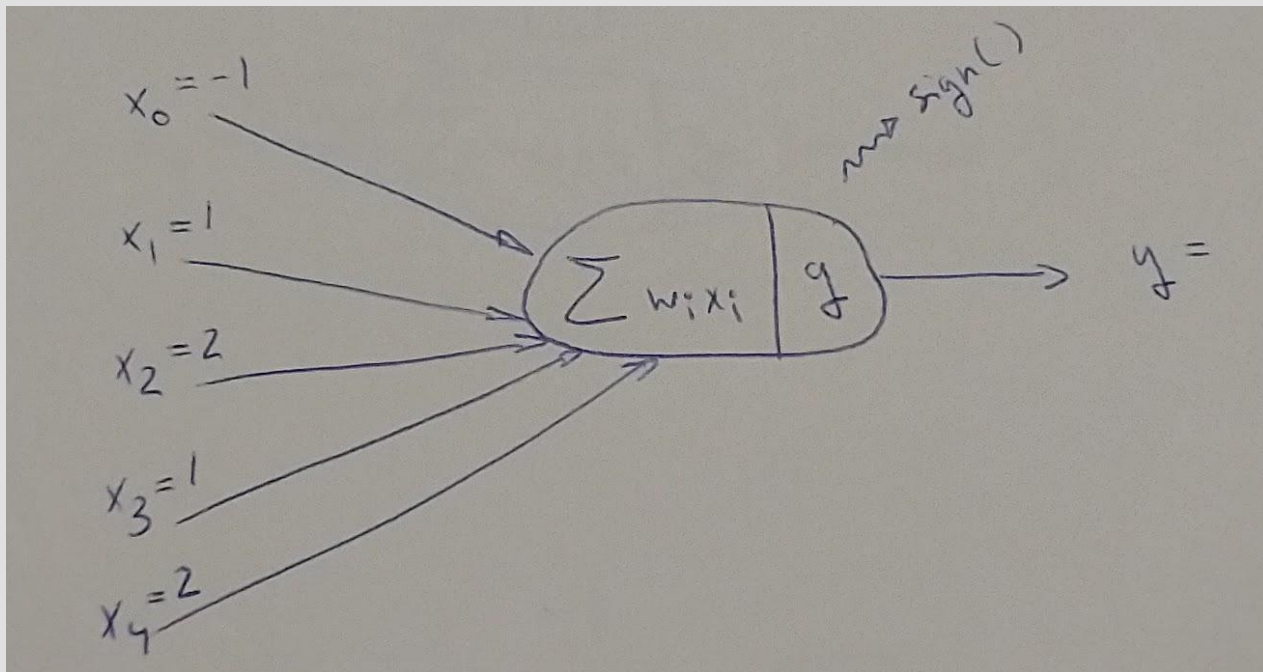
Nonlinear functions are better (later we need derivatives)

The Perceptron - Example

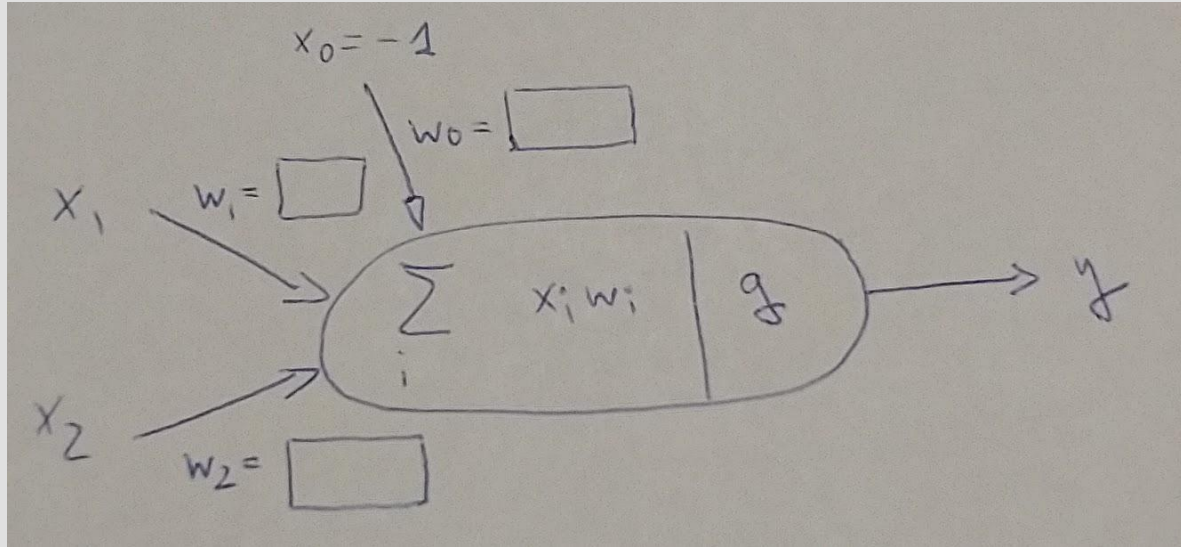


The Perceptron

- We can get rid of the bias without complicating the model
 - additional input set to -1 (constant), weight set to the bias



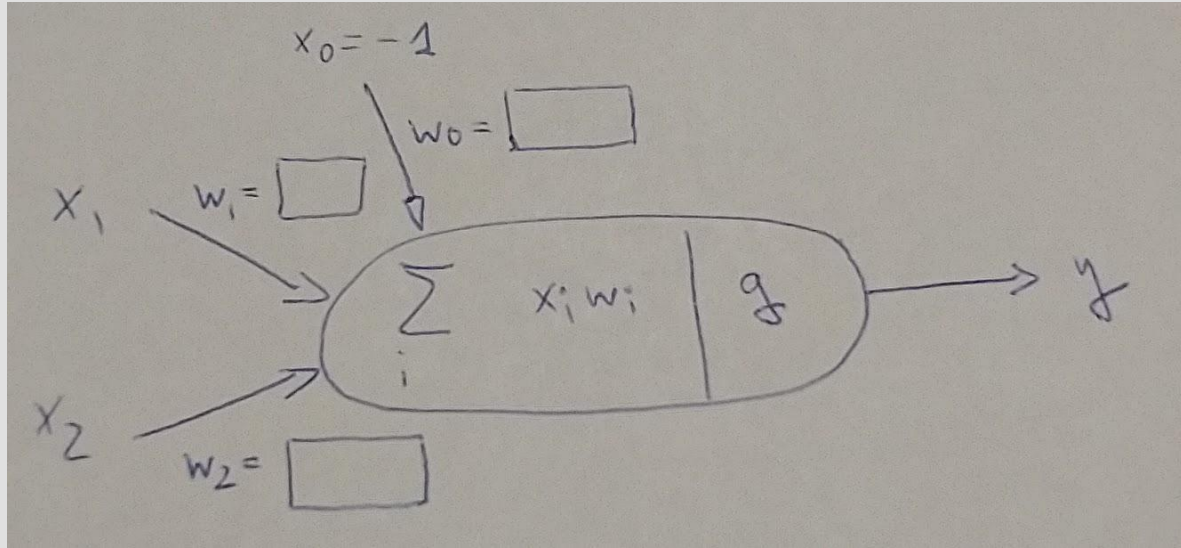
The Perceptron - AND operator



x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

- What does the output equal to?
- How does the dataset and the model look like?
- What is the “right” w_0 ?

The Perceptron - OR operator

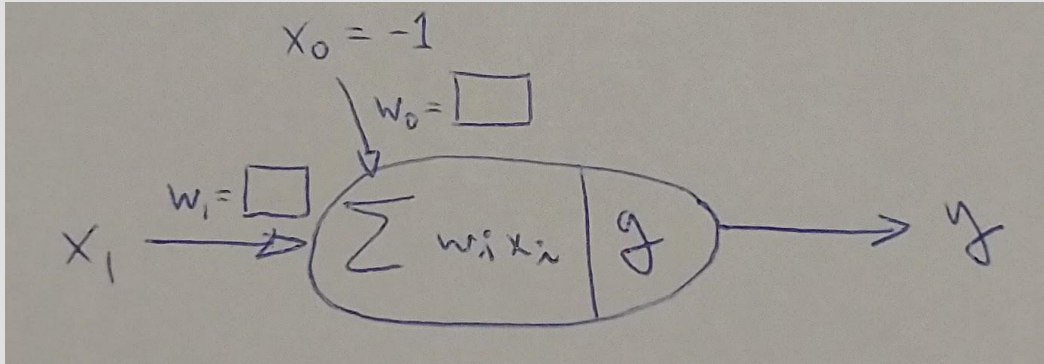


x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

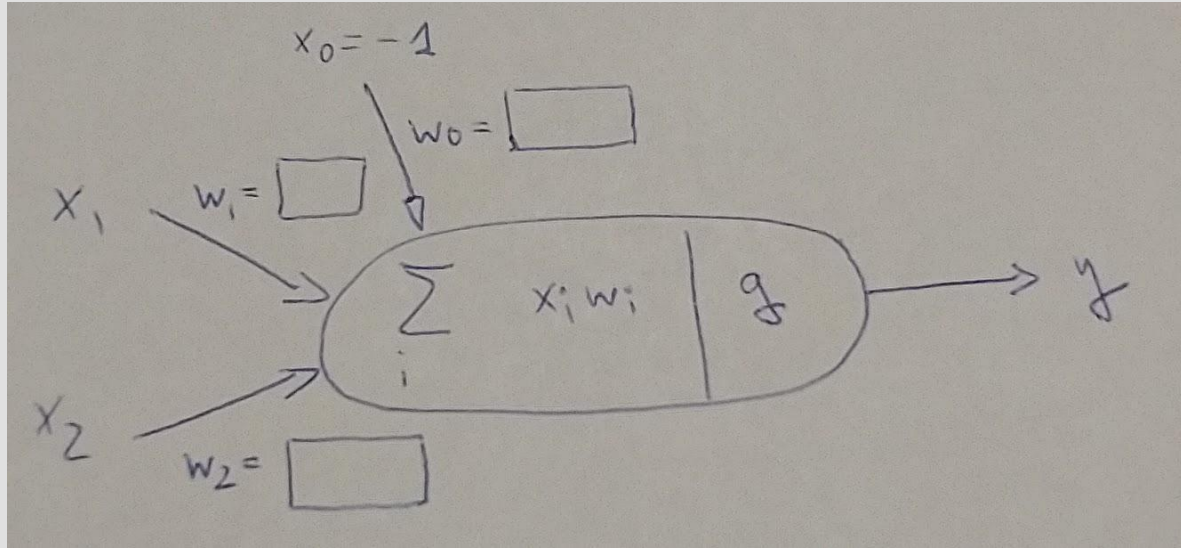
- What does the output equal to?
- How does the dataset and the model look like?
- What is the “right” w_0 ?

The Perceptron - NOT operator

x_1	y
0	1
1	0

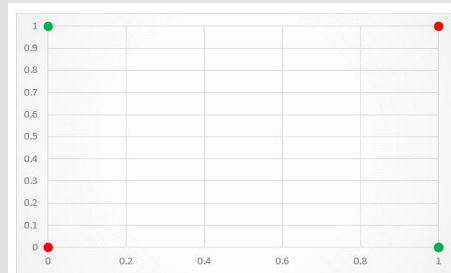


The Perceptron - XOR operator



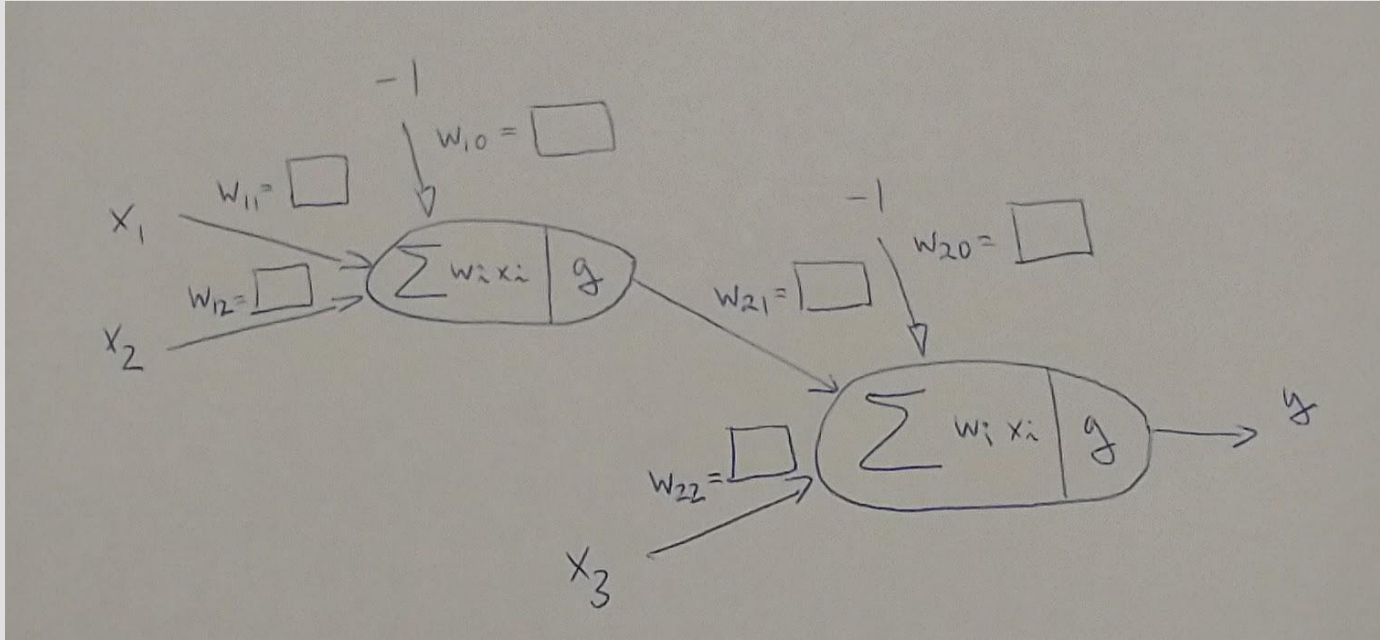
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

- It is not linearly separable!



The Perceptron - Complex Functions

$$y = (x_1 \text{ OR } x_2) \text{ AND } x_3$$



The Perceptron - XOR revisited

$$y = x_1 \text{ XOR } x_2$$

Exercise: Design a network (1 hidden layer) to calculate the XOR.

The Perceptron - Fitting Parameters

- We want to learn the weights of the perceptron
 - architecture (aka topology) is fixed (one neuron)
- Main ideas:
 - Goal: Adjust the weights to minimize the error in the training set.
 - Calculate the error comparing the output of the perceptron and the correct class
 - High-level steps:
 - 1. Run one example through the network
 - 2. Error?
 - No: pass; no need to update weights
 - Yes: adjust weights to make the error smaller
 - 3. Go to 1

The Perceptron - Fitting Parameters

- Main ideas:
 - Goal: Adjust the weights to minimize the error in the training set.
 - Calculate the error comparing the output of the perceptron and the correct class
 - High-level steps:
 - 1. Run one example through the network
 - 2. Error?
 - No: pass; no need to update weights
 - Yes: adjust weights to make the error smaller
 - 3. Go to 1
- Some terminology:
 - online learning (look at one example at a time)
 - batch: bunch of examples used to calculate error (and update weights accordingly)
 - epoch: one cycle through all the training examples
 - num. epochs > 1 (fitting the weights for one example may mess them up for another one)

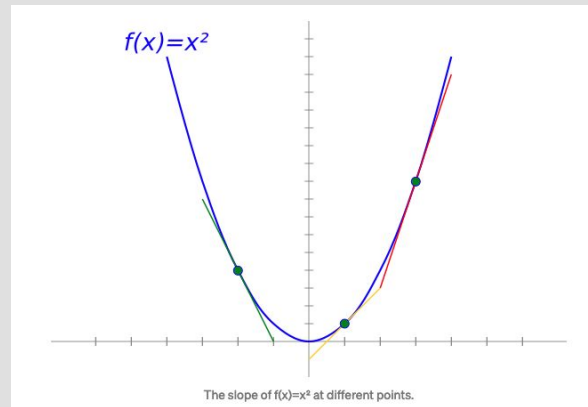
The Perceptron - Fitting Parameters

- How do we update the weights?
 - The error depends on the weights (the training dataset is fixed and given to us)
 - coordinates and labels
 - Minimizing the error is key
 -

- Gradient of a function:

$$f(x_1, x_2, \dots, x_n) = \nabla f = \left[\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2}, \dots, \frac{\partial}{\partial x_n} \right]$$

- The gradient points in the direction of greatest increase
- We want to move against (opposite to) the gradient of the error function
 - because we want to minimize the error



The Perceptron - Fitting Parameters

- Defining the error function
- Calculating the gradient and the weight update rule
- Details:
 - Learning rate is usually set low (and ideally lower it as more weight updates are made)
 - Weight initialization: all zero, random, more sophisticated methods
 - Activation function: step vs. sigmoid (not differentiable vs. differentiable)